

实验一：基于 YOLOv8 与 Jetson 的四类桌面物体检测系统

机器人学集成小组项目
中国海洋大学计算机科学与技术学院

个人 GitHub: <https://github.com/9161942-boop/robotics-experiment-1>

Abstract—本文报告一个面向桌面物体的端到端目标检测系统。实验从桌面物体视频抽帧、样本筛选和 ISAT 人工标注开始，经类别映射与 YOLO detection 格式转换，在四类自定义数据上训练 COCO 初始化的 YOLOv8n，并将最佳权重部署到 Jetson Orin。数据集包含 595 张图像，按来源时间块划分为 421 张训练图像、87 张验证图像和 87 张测试图像；模型在验证集上的 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95 分别为 0.658、0.748、0.766 和 0.701。系统同时保留 mouse、laptop、cup、phone 四类输出，能够叠加显示类别、边界框、置信度和 FPS，并通过 ROS2 发布 /detections。在完全独立、每类 20 张的 80 张实物照片上，系统通过 72 张，图片级准确率为 90%；Jetson 演示视频显示约 15 FPS。报告进一步说明独立测试判据、典型错误、部署参数、复现命令和 GitHub 分阶段提交证据。

Index Terms—目标检测，YOLOv8，Jetson，ROS2，独立实物测试，实验报告

I. 实验目标与系统概览

本实验的目标不是只得到一个离线精度数字，而是建立一套可运行、可复现、可验收的桌面物体检测链路。系统输入为 USB 摄像头的连续图像，核心模型为四类 YOLOv8n，输出包含类别、边界框和置信度，并在 Jetson 上实时显示。训练侧负责从原始视频恢复可用样本，部署侧负责把相同的权重和推理参数迁移到边缘设备，接口侧再将检测结果封装为 ROS2 消息。完整链路为：摄像头采集 → YOLOv8 检测 → Jetson 实时显示 → ROS2 发布。

实验选择 mouse、laptop、cup 和 phone 四类常见桌面物体。这样既满足“不少于两类”的课程要求，又能覆盖小目标、细长目标、反光表面和不同遮挡程度等情况。最终验收分为三个层面：带标注数据上的模型指标用于判断训练是否收敛；完全独立的实物照片用于判断跨拍摄条件的识别可靠性；Jetson 视频和程序文件用于证明系统能够从模型走向实时应用。

验收结果：独立实物照片测试 72/80 (90%)，高于 80% 门槛；Jetson 视频画面显示约 15 FPS，高于 5 FPS 门槛；数据集、模型、程序、视频、运行说明和错误案例均已归档。

II. 完整实验流程

实验遵循“采集—筛选—标注—转换—训练—验证—独立测试—部署—验收—归档”的顺序。每一步均产生可追溯文件：原始图片和 ISAT JSON 保留采集与标注依据，YOLO 目录保留训练输入，训练目录保留曲线、参数和权重，独立测试目录保留逐图结果与失败样本，Jetson 目录保留部署脚本和演示视频，最终通过 GitHub 分阶段 commit 固化。



图 1. 从原始视频到可验收系统的完整实验流程

- 1) 从桌面物体视频中抽帧，去除过暗、严重模糊和目标不完整的画面，再保留不同角度、距离和背景的本样。
- 2) 使用 ISAT 为每个可见目标绘制紧致边界框，统一使用四个英文类别名，并逐张检查框是否越界、漏标或错标。
- 3) 将 ISAT JSON 转换为 YOLO detection 格式，即每行一个目标的类别编号与归一化中心坐标、宽度和高度。
- 4) 按来源时间块建立 train/val/test 划分，避免相邻视频帧同时出现在训练和评估集合中，并检查图像与标签一一对应。
- 5) 以 COCO 预训练 YOLOv8n 为初始化训练 100 个 epoch，记录参数、随机种子、曲线、混淆矩阵和最佳权重。
- 6) 在验证集和 test 诊断集上计算 Precision、Recall、mAP，并结合可视化结果定位漏检、误检和类别混淆。
- 7) 使用完全未参与训练的 80 张独立实物照片进行图片级筛查，按“目标是否被正确识别”统计每类及总体通过率。
- 8) 将四类权重部署到 Jetson Orin，运行摄像头程序，检查检测框、类别、置信度、物体数量和滑动平均 FPS。

9) 通过 ROS2 节点把图像检测结果封装为 `Detection2DArray` 并发布到 `/detections`，最后整理报告、材料清单和 GitHub 记录。

III. 硬件、软件与数据集

A. 实验环境

表 I. 实验环境与数据配置

训练平台	Windows 11 + RTX 4060 Laptop GPU
部署平台	Jetson Orin, Ubuntu 20.04
框架与版本	YOLOv8n, Ultralytics 8.4.22
ROS2	Humble, 消息为 <code>Detection2DArray</code>
输入图像	640×480 USB 摄像头
类别与规模	mouse / laptop / cup / phone; 图像总数 595 (421/87/87)

训练机使用 Windows 11 和 RTX 4060 Laptop GPU，部署机为 Jetson Orin，系统为 Ubuntu 20.04。训练和部署均采用 Ultralytics YOLOv8n，ROS2 接口使用 Humble 版本的标准消息类型。摄像头输入按 640×480 采集；Jetson 推理时将图像缩放到 320 作为速度和精度之间的折中，输出视频仍保留原始画面比例。

B. 数据采集与划分

数据来自桌面物体视频抽帧，类别为 mouse、laptop、cup 和 phone，共 595 张图像。训练集、验证集和测试集分别为 421、87 和 87 张。划分依据恢复出的 19 个实物或视频来源，并在每个来源内部按时间块分配到不同集合，因而相邻帧不会简单地同时出现在训练集和评估集。训练只使用 `train/val`，`test` 集保留到训练结束后用于诊断，保证最终模型选择不直接依赖测试集。

C. 标注与格式转换

每张原始图像先在 ISAT 中人工绘制边界框并保存 JSON 标注，再由转换脚本写入 YOLO 标签。转换后的每一行遵循“`class_id x_center y_center width height`”格式，坐标均相对于图像宽高归一化到 $[0, 1]$ 。类别编号由 `data.yaml` 统一管理，避免训练和推理时出现名称错位。转换后对图像数量、标签文件数量、类别编号范围和空标签情况进行检查，并将检查后的目录作为训练输入。图 2 给出四类样本及其人工边界框。



图 2. 数据集样本与人工标注框示例

IV. 训练方法与模型质量

A. 训练配置

以 COCO 预训练 YOLOv8n 为初始化，在四类数据集上训练 100 个 epoch，输入尺寸 640，batch size 16，随机种子 42，workers 为 0，使用自动优化器和 AMP 混合精度。训练阶段启用 mosaic、随机翻转、缩放和 RandAugment 等增强，最后 10 个 epoch 关闭 mosaic 以稳定收敛。验证阶段使用 Ultralytics 默认 NMS，离线可视化置信度阈值为 0.25；Jetson 实时演示提高到 0.35，以减少低置信度框对画面的干扰。训练配置、曲线和最佳权重保存在仓库中，最终模型为四类输出的 `weights/best.pt`。

B. 指标与结果解释

Precision 衡量预测框中真正例的比例，Recall 衡量标注目标被找回的比例，`mAP@0.5` 和 `mAP@0.5:0.95` 则分别在单一 IoU 阈值和多个 IoU 阈值下汇总各类别平均精度。验证集指标用于观察模型泛化和选择最佳权重，`test` 诊断结果用于检查未参与训练的帧级表现；二者都依赖人工边界框。独立照片测试没有额外人工框，因此只统计图片级“是否正确识别”，不能将其与 `mAP` 直接等同。

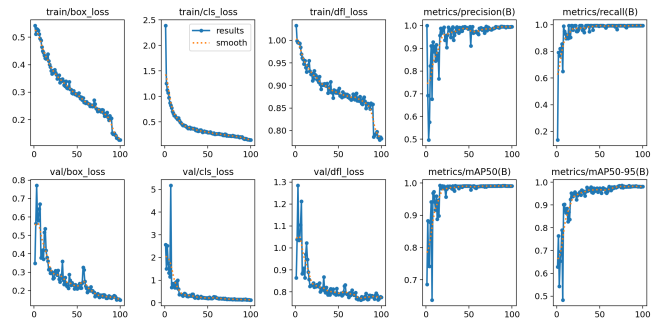


图 3. 训练过程指标曲线（对应仓库 `results/training/results.png`）

表 II. 验证集与诊断集结果

数据/指标	P	R	mAP@50	mAP@50:95
dataset/yolo 验证集	0.658	0.748	0.766	0.701

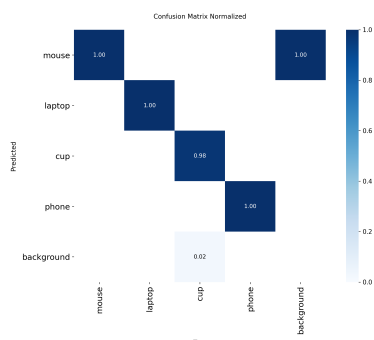


图 4. 四类目标的归一化混淆矩阵

独立验收集共 80 张照片，每个类别 20 张，按“该照片中目标是否被正确识别”统计：cup 17/20、laptop 19/20、mouse 18/20、phone 18/20，总计 72/80 (90%)。未通过样本为 04_cup.jpg、12_cup.jpg、13_cup.jpg、21_laptop.jpg、42_mouse.jpg、46_mouse.jpg、72_phone.jpg、80_phone.jpg。

80 张照片来自训练数据之外的独立实物集合，每类固定 20 张，尽量覆盖不同机身、摆放方向、距离和背景。判定规则是：目标类别预测正确且至少有一个检测框覆盖目标主体，则该图片通过；漏检、类别错误或只检测到背景均记为未通过。该规则与课程“测试 20 个物体、正确率不低于 80%”的现场检查方式一致，并额外给出四类的分项结果，便于定位薄弱类别。cup 的通过率为 85%，laptop 为 95%，mouse 和 phone 均为 90%。

表 III. 独立实物照片的类别级通过统计

类别	测试数	通过数	通过率
cup	20	17	85%
laptop	20	19	95%
mouse	20	18	90%
phone	20	18	90%
总计	80	72	90%

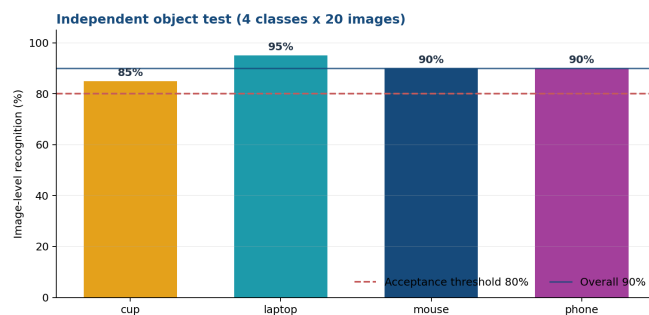


图 5. 80 张独立实物照片的分类型通过率与 80% 验收门槛

V. JETSON 部署与 ROS2 发布

A. 实时推理

Jetson 演示视频分辨率为 640×480 ，程序从摄像头读取连续帧，在 GPU 上完成 YOLOv8n 推理和 NMS，再把检测框、类别、置信度、物体数量和滑动平均 FPS 叠加到画面。原视频中的 FPS 显示约 15.0，满足不低于 5 FPS 的验收要求；抽帧中可同时看到 mouse、phone、laptop 和 cup 的检测框。部署脚本支持通过 SSH 上传代码和指定权重，现场只需修改摄像头设备号或实际图像话题即可复用。

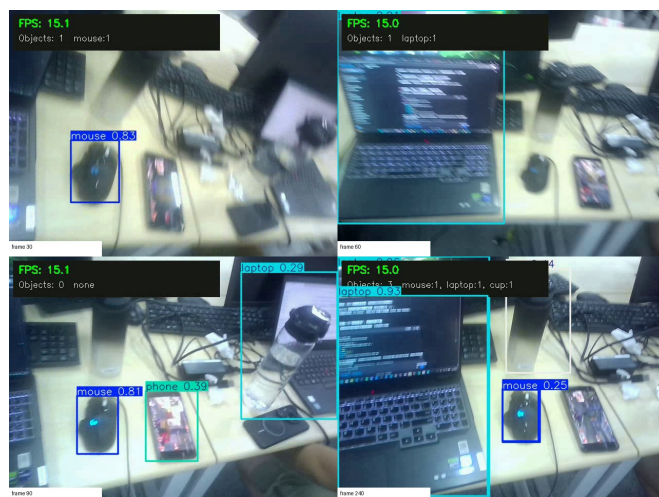


图 6. Jetson 演示视频抽帧（原视频：results/videos/jetson_demo.mp4）

B. ROS2 数据链

检测节点不直接创建摄像头驱动，而是订阅相机节点发布的 `sensor_msgs/Image`。节点对每帧执行与实时程序相同的预处理、推理和置信度筛选，然后将每个目标封装到 `vision_msgs/Detection2DArray` 中：消息头记录时间戳，检测结果包含类别名、置信度和像素坐标框。检测节点订阅 `/camera/image_raw`，推理后发布 `/detections`。程序、启动方式、参数和验证命令写入 docs/ros2_quickstart.md，现场验证命令为：

```
ros2 topic echo /detections --once
```


VI. 错误分析与可复现性

4. 典型错误

错误案例已保存于 `results/independent_test_80/errors/` 和 `results/evaluation_final/errors/`。8 张未通过照片中，`cup` 失败 3 张，主要表现为杯体边缘被桌面背景吞没；`laptop` 失败 1 张，与斜视角和屏幕反光有关；`mouse` 失败 2 张，与目标尺寸较小或被遮挡有关；`phone` 失败 2 张，与新机型外观、低对比度和摆放方向变化有关。总体来看，失败主要与小目标、运动模糊、遮挡、反光和低置信度有关，而不是类别数量不足。

B. 复现与改进

在 Windows 训练机中，先运行 `python convert_isat_to_yolo.py` 生成或更新 `dataset/yolo/`，再运行 `python train_yolo.py --epochs 100 --batch 16 --device 0 --workers 0`。评估可运行 `python evaluate_yolo.py --device 0`；独立照片的逐图结果位于 `results/quality_4class_best_independent/`。最终权重的 SHA-256 完整值记录在仓库 README 中，报告中保留其首尾校验片段：

55341D55082158F799C0DA7ED813C959
0EE2AE043ED2AD837DBD0B47CB0E3FE4

可用该摘要确认下载文件未被替换。后续工作可补充困难样本和遮挡增强，提高小目标输入尺寸，并使用 TensorRT 进一步压缩延迟；这些方向属于改进计划，不改变本次验收结果。

VII. GitHub 提交记录与材料追踪

按照课程要求，报告、数据、代码、模型和结果均上传至个人 GitHub，并保留分阶段 commit 记录，未采用一次性全量提交。仓库链接为：<https://github.com/9161942-boop/robotics-experiment-1>。

表 IV. 代表性分阶段提交

阶段	Commit	内容
数据准备	5fcd148	鼠标样本与标注计划
数据准备	c2507b4	cup/laptop/phone 数据补充
训练管线	3e007d8	四类 YOLO 训练流程
独立测试	a93ffb5	独立图片筛查记录
Jetson 部署	14a710c	实时检测演示视频
验收整理	d887607	验收材料与 PDF 报告

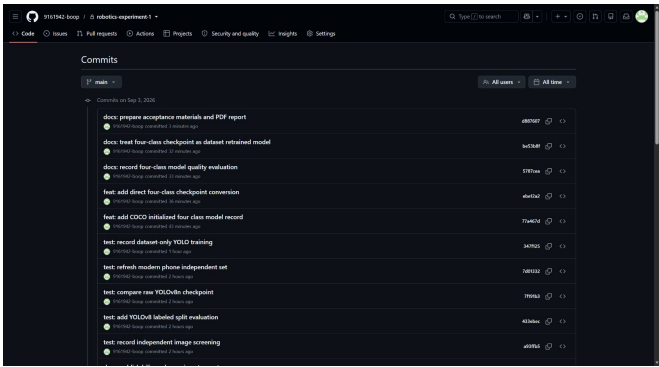


图 7. GitHub main 分支提交记录截图（页面访问时间：2026-09-03）

VIII. 提交材料与验收结论

提交材料包括：数据集 `dataset/images/`、`dataset/yolo/`；四类模型 `weights/best.pt`；程序 `src/`、`scripts/`；Jetson 视频 `results/videos/jetson_demo.mp4`；运行说明 `README.md`、`docs/ros2_quickstart.md`；训练曲线、混淆矩阵、独立测试统计图、失败样本和报告源文件。材料目录同时保留原始数据、可直接训练的数据和最终验收结果，便于教师按“数据—模型—程序—视频—报告”的顺序检查。所有材料按目录整理，并通过 GitHub 分阶段 commit 留存。

表 V. 验收要求与材料证据对照

验收要求	对应证据	状态
同时识别不少于 2 类	四类模型（mouse、laptop、cup、phone）及 80 张独立照片测试记录	达成
20 个物体正确率不低于 80%	独立测试汇总：72/80，准确率 90%，并附类别统计图	达成
Jetson 实时速度不低于 5 FPS	<code>results/videos/jetson_demo.mp4</code> 中叠加显示约 15 FPS	达成
保存结果与典型错误案例	<code>results/independent_test_80/</code> 、 <code>results/evaluation_final/</code> 及错误样例目录	达成
ROS2 发布识别结果	节点发布 <code>/detections</code> ，启动与 <code>ros2 topic echo</code> 命令写入运行说明	已实现

结论

本实验完成了从数据采集、标注、YOLOv8 训练到 Jetson 实时检测和 ROS2 发布的完整链路。四类独立实物照片准确率为 90%，Jetson 视频显示约 15 FPS，达到本实验的主要验收指标。论文式报告、源文件、数据、模型、程序、视频、运行说明和错误案例均已整理并上传 GitHub。